



EXPERIMENT: 10

Name:- Ravi

Branch:- BE-CSE

Semester:- 6th

Subject Name:- System Design

UID:- 23BCS10340

Section:- KRG 2-A

Date :- 15-04-2026

Subject Code:- 23CSH-314

1.Aim:-

To design a scalable Live Voting System (similar to polls on social platforms) that supports high concurrent voting, real-time results, and ensures no vote loss under heavy traffic using React, Spring Boot, and PostgreSQL.

2.Objectives:-

1. To understand concurrency issues in high-traffic systems.
2. To design APIs for vote submission and result retrieval.
3. To study optimistic and pessimistic locking techniques.
4. To analyze real-time data handling using caching.
5. To prevent race conditions during simultaneous updates.
6. To build a minimal yet scalable backend system.

3.Procedure:-

1. Designed a simple poll system with two options (e.g., Tabs vs Spaces).
2. Identified key entities: Users, Polls, Votes.
3. Defined
User → Submit Vote → Store Vote → Update Count → Fetch Results
4. Implemented REST APIs for voting and result fetching.
5. Analyzed concurrency issue when multiple users vote simultaneously.
6. Applied **Optimistic Locking** in PostgreSQL to avoid lost updates.
7. Used **Redis (optional)** for caching results.
8. Ensured atomic updates using transactions.
9. Tested system with concurrent requests.

workflow:

4. Functional Requirements:-

- i. Users should be able to view polls.
- ii. Users should be able to vote for an option.
- iii. System should store each vote.



- iv. System should display real-time vote count.
- v. Each user can vote only once (optional constraint).

5.Non Functional Requirements:-

- i. System should handle high concurrent traffic (1000+ requests/sec).
- ii. Vote submission must ensure **strong consistency**.
- iii. API latency:
 - Vote API < 200 ms
 - Result API < 100 ms
- iv. System should be scalable.
- v. Use caching to reduce database load.
- vi. Ensure fault tolerance.

6.Core Entities:-

- User
- Poll
- Option
- Vote

7.API Design:-

- i. Poll APIs:
 - GET /api/v1/polls
- ii. Voting APIs:
 - POST /api/v1/vote
- iii. Result APIs:
 - GET /api/v1/results/{poll_id}

8.Database Schema Design:-

- **i. Users Table**
 - user_id (PK)
 - name
 - email
- **ii. Polls Table**
 - poll_id (PK)
 - question
- **iii. Options Table**
 - option_id (PK)
 - poll_id (FK)
 - option_text
 - vote_count
 - version (for optimistic locking)
- **iv. Votes Table**

- • vote_id (PK)
- user_id (FK)
- option_id (FK)
-

9.HLD:-

